

# Project 3 — DocuMind-Smart-Documents-Agent-Ask-Your-Documents-Anything-Safely

---

**Title:** DocuMind Smart Document Agent Ask Your Documents Anything Safely

**Type:** Multi-agent (3 agents) · **Stack:** Python, LangGraph, RAG (Chroma/FAISS) · **Level:** Intermediate **Tagline:** A 3-agent LangGraph system — Retriever, Reasoner, and Approval — that grounds every answer in your documents and pauses for a human before acting.

The full build spec lives in [10\\_agent\\_workflows/README.md](#) (building on [09\\_langgraph](#)). This file is your workspace and checklist, not a second copy of that spec.

## Charter (what this project is)

---

Project 2's Worker agent, rebuilt as a clear, checkable LangGraph, then grown into three agents working together: a Retriever (searches + judges its own results), a Reasoner (answers only from what was found), and an Approval agent (checks automatically, and only asks a human via `interrupt()` when it really needs to). This is where "agent" becomes "graph," and "one agent" becomes "a small team built right into the graph."

**Jump to:** [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#) · [Step 5](#)

## The Story — what you're actually building

---

Imagine you've built a support agent that can look things up and act (that's Project 2). Now imagine someone asks it a question about *your own documents* — a policy PDF, an internal wiki page, a contract — not something the model already knows from training. And imagine the action it's about to take is actually risky: sending a real email, issuing a refund, deleting a record. You want the agent to only answer from what it actually found in your documents, and you want a human to get a say before anything risky happens — but not on *every single run*, or nobody would ever use it.

That's exactly what you're building here, as three agents working in sequence. The **Retriever** is the one who goes and looks: given a question, it searches your documents and — this part matters — honestly judges whether what it found is actually good enough to answer from, instead of just handing back "here's the closest thing I found" and hoping. The **Reasoner** is the one who writes the answer, and only from what the Retriever actually found — it's not allowed to fill gaps with things it merely remembers. The **Approval** agent is the last check before anything happens: it runs its own

automatic checks on the Reasoner's draft, clears the safe, well-grounded ones on its own, and only pauses to ask a real person when its checks aren't confident or the action is genuinely important.

Underneath all three, the whole thing is rebuilt as a **graph** instead of a hand-written loop, because a graph is something you can actually draw, pause, save, and resume — which a loop buried in a function can't easily do. That's the other half of this project: not just "three agents," but agents whose state is visible and checkable at every step, and that can be paused mid-run and picked back up later without losing anything.

The Steps below build this up in order: Step 1 proves the barest graph mechanics work — state, nodes, one edge — with nothing smart in it yet. Step 2 takes Project 2's already-working agent and rebuilds it as that kind of graph, gaining saved state and a pause/resume cycle for free. Step 3 adds the Retriever and its self-judgment. Step 4 adds the Reasoner, which only uses what Step 3 found. Step 5 adds the Approval agent and its `interrupt()` gate, completing the full search → answer → approve/pause → resume cycle.

**What you're actually building, in one line:** a 3-agent LangGraph pipeline — a Retriever that searches your documents and judges its own results, a Reasoner that answers only from what was found, and an Approval agent that pauses for a human before anything risky.

**Why this needs to exist:** a model answering from training data alone can sound confident about your specific documents while being completely wrong about them, and letting an agent take a real action — send an email, issue a refund — with no human check at all is genuinely risky.

**When you'd reach for this at a real job:** when answers need to be grounded in your own private documents (a policy, a contract, an internal wiki) instead of the model's general knowledge, and when at least one downstream action matters enough to need a person's sign-off, but not so often that a human has to check every single run.

**How it works, mechanically:** LangGraph's `StateGraph` carries one typed state object through the Retriever, Reasoner, and Approval nodes; a conditional edge routes around the Reasoner when the Retriever's search wasn't good enough, and `interrupt()` pauses the graph and saves its state until a human responds.

**Why not just do it some simpler/different way:** why not just paste all your documents straight into the prompt instead of retrieval? Because a real set of documents is far bigger than any context window, and stuffing everything in wastes tokens on parts that have nothing to do with the question. Why not just always ask a human before every action, or never ask at all? Always-asking doesn't scale — nobody wants a tool that stops for approval on every trivial case — and never-asking means a wrong, risky action can go out with nobody watching.

## Why This Project Is Good for Your Portfolio

---

- **What problem it solves:** shows an agent whose inner state and decisions are checkable and can be paused/resumed, not a black box — you can point at the exact graph diagram and explain exactly what happens at every step, including pausing for a human before a risky action.
- **Why it matters:** this is the jump from "a script that happens to work" to "a system someone could actually run in production" — clear state, branching, saved state, and human approval are exactly what a hiring manager is checking for when they ask "how would you make this safe to actually use."
- **When you'd build something like this at a real job:** any agent that needs to survive a restart, needs a human check before something important (sending something, spending money, deleting data), or needs to ground its answers in your own documents.
- **How it's built:** a `StateGraph` with typed state, a search step that only runs when needed, saved state, and an `interrupt()` gate before the risky action — a full search → answer → pause → resume → answer cycle you can draw straight from the code.

### Problems you'll likely run into, and how to fix them:

| Problem                                                                               | Fix                                                                                                                                                      |
|---------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| The graph loops forever, because a branch has no real exit rule                       | Treat every loop in the graph the same way you'd treat Project 2's step limit — design and test the exit rule on purpose, don't assume it'll "just work" |
| The state saved at a pause doesn't include what was found, only the original question | Include the found chunks directly in the state shape, so resuming doesn't need to search again or lose its grounding                                     |
| Search finds nothing relevant, and the graph answers anyway                           | Route to a clear "can't ground this answer" path when the search results are below a set quality bar, instead of letting it answer without support       |

## Setup (do this once, before Step 1)

```
cd project_3_documind
python -m venv .env
source .env/bin/activate
pip install openai langchain langchain-openai langgraph chromadb python-dotenv py
pip freeze > requirements.txt
```

New here vs. Project 2: `langgraph` (the graph tool), `chromadb` (a local vector store for RAG — swap for `faiss-cpu` if you prefer FAISS). Same `.env` setup as before.

**What to put in the vector store:** you need real text to search before you can test retrieval. A good option is the Hugging Face dataset [rag-datasets/rag-mini-wikipedia](https://huggingface.co/datasets/rag-datasets/rag-mini-wikipedia) — it's small, public domain, and built specifically for practicing RAG pipelines like this one, so it embeds fast and runs

locally without a huge download. This is a convenience, not a requirement — your own text files (notes, a PDF, a policy doc) work just as well.

## Built During These Documents

---

[08\\_rag](#) → [Async Python gate](#) → [09\\_langgraph](#) → [10\\_agent\\_workflows](#)

## Plan Before You Code

---

See [15\\_five\\_projects\\_index](#): write out Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks — specifically, sketch the graph's node/edge diagram on paper before writing `graph.py`.

## How To Build This — Step by Step

---

### Step 1 — A 2-Node Graph With No Branching, Just to Prove the Mechanics

*Project: DocuMind-Smart-Documents-Agent-Ask-Your-Documents-Anything-Safely — Step 1 of 5: A 2-Node Graph With No Branching, Just to Prove the Mechanics*

**What this step does:** proves the basic graph mechanics — state, nodes, one edge — actually work, before any agent behavior or branching gets added. Keeps "do I understand the graph basics" separate from everything after it. **Why this step matters:** every step after this one adds nodes and branches to this exact skeleton — if you don't trust that state actually flows from node to node here, you can't trust it once branching and agents make the graph harder to read. **When you'll hit this for real:** the start of every new LangGraph project you'll ever build — always prove the skeleton compiles and runs before adding real logic to it. **Read first:** [09\\_langgraph Core Concepts](#) — "From a hidden loop to a clear graph," "State," "Nodes and edges."

**Stuck on this step?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Build a minimal 2-node `StateGraph` with a typed state shape and one plain edge — no tools, no branching yet (this is Doc09's own "Basic" exercise). Define the state as a `TypedDict` (or Pydantic model) with named fields up front, even though it's tiny now — every later step in this project only ever adds fields to this same shape, it's never replaced. 2. By hand, trace what the state looks like after each node runs. Check that your own understanding matches what the code actually does, before adding anything else.

**Your files after Step 1:**

```
project_3_documind_rag_agent/
├─ state.py           → a minimal TypedDict/Pydantic state
```

```

├─ nodes.py          → 2 simple node functions
├─ graph.py          → build_graph() with 2 nodes, 1 edge, no branching

```

## Step 2 — Project 2's Agent, Rebuilt as a Graph With Saved State

*Project: DocuMind-Smart-Documents-Agent-Ask-Your-Documents-Anything-Safely — Step 2 of 5: Project 2's Agent, Rebuilt as a Graph With Saved State*

**What this step does:** proves your existing agent (from Project 2) can be rebuilt as graph nodes with the exact same behavior, and now gains saved state. **Why this step matters:** this is the migration you'll actually do at a real job far more often than a from-scratch build — taking something that already works and moving it onto infrastructure (here, a graph with a checkpoint) that makes it pausable and inspectable, without changing what it does. **What's new vs. Step 1:** real nodes (Project 2's tool-calling logic) replace the simple placeholder nodes; a conditional edge and a checkpoint get added. **What stays the same:** the tools themselves and the Worker's decisions — you're changing the *container* (loop → graph), not the agent's behavior. **When you'll hit this for real:** any time you outgrow a hand-rolled agent loop and need it to be pausable, resumable, or inspectable — this exact migration, not a rewrite from scratch. **Read first:** [09\\_langgraph Core Concepts](#) — "Loops," "Checkpoints," " `interrupt()` ".

**Stuck on this step?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Rebuild Project 2's Worker agent (tool-calling loop) as graph nodes with a conditional edge — same tools, same behavior, now built as a graph (Doc09's "Real-world" exercise). 2. Connect a checkpoint and prove a pause/resume cycle works on this single-agent graph, before adding any more agents. Test the resume in a fresh process, not just the same one still running — a checkpoint that only appears to work because the Python objects are still sitting in memory isn't actually proving the state was saved.

**Your files after Step 2:**

```

project_3_documind_rag_agent/
├─ state.py
├─ nodes.py          → single-agent tool-calling nodes (from Project 2)
├─ graph.py          → conditional edge + checkpoint connected
├─ tools.py          → reused from project_2_researchhand

```

## Step 3 — A Retriever Agent That Judges Its Own Search Results

*Project: DocuMind-Smart-Documents-Agent-Ask-Your-Documents-Anything-Safely — Step 3 of 5: A Retriever Agent That Judges Its Own Search Results*

**What this step does:** adds the first new specialist — a node that doesn't just fetch chunks, but decides whether what it found is actually good enough to answer from. **Why this step matters:** a

RAG system that always hands its search results to the answerer, good or bad, is how you get a confident answer built on nothing — judging relevance before answering is what actually earns the word "grounded." **What's new vs. Step 2:** a new Retriever node appears, only run when needed; the state grows to carry found chunks, not just the question. **What stays the same:** Step 2's checkpoint and the core graph-building pattern — you're adding a node to an already-proven base, not rebuilding it. **When you'll hit this for real:** any "answer questions about my documents" feature request — this is the exact node that turns a plain agent into a RAG agent. **Read first:** [08\\_rag Core Concepts](#) — "Vector stores and searching for the top matches."

**Stuck on this step?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Build the **Retriever agent** as its own node: wraps `08_rag`'s `retrieve()`, plus a step that judges relevance (is what came back actually useful? if not, route to a "can't ground this" path instead of forcing an answer downstream). Write the relevance bar down as an explicit, checkable rule (a similarity-score cutoff, or a concrete judged question like "do these chunks actually mention the thing being asked?") before you test — a vague "does this look okay?" bar just gets adjusted after the fact to match whatever the first test happened to do. 2. Wire it in as a conditional step in the Step 2 graph — some tasks skip it entirely, same idea as Doc10's routing. 3. Test it: a question the knowledge base can answer, and one it can't — confirm the "can't ground this" path actually runs on the second one.

**Your files after Step 3:**

```
project_3_documind_rag_agent/
├─ state.py                → now includes found chunks, not just the question
├─ graph.py                → conditional routing to the Retriever node
├─ retriever.py            → reused from 08_rag
├─ agents/
│   └─ retriever_agent.py  → search + relevance-judging node
└─ nodes.py
```

## Step 4 — A Reasoner Agent That Only Answers From What the Retriever Found

*Project: DocuMind-Smart-Documents-Agent-Ask-Your-Documents-Anything-Safely — Step 4 of 5: A Reasoner Agent That Only Answers From What the Retriever Found*

**What this step does:** adds the specialist that actually writes an answer, and only from grounded material — keeping "did we find good context" (Step 3) separate from "did we use it correctly" (this step). **Why this step matters:** search quality and answer quality fail for different reasons and need different fixes — separating them into two nodes means when an answer is wrong, you can tell in seconds whether the Retriever found the wrong thing or the Reasoner used the right thing badly. **What's new vs. Step 3:** a new Reasoner node uses the Retriever's output; the graph now has search → answer as a real sequence. **What stays the same:** the Retriever from Step 3 is untouched — Step 4 just adds something that uses its output. **When you'll hit this for real:** the

moment a RAG system's answers start drifting from what was actually retrieved — this separation (search quality vs. answer quality) is exactly how you'd diagnose and fix that. **Read first:** [10\\_agent\\_workflows Core Concepts](#) — how the pieces fit together.

**Stuck on this step?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Build the **Reasoner agent**: takes the Retriever's output plus the task, and writes a grounded draft answer — clearly told not to answer beyond what was found. Spell out explicitly what it should say when the found chunks don't actually cover the question — an unprompted model tends to quietly fill the gap from what it remembers rather than admit the retrieved material falls short. 2. Wire search → answer as a real edge sequence. 3. Test it: check that the draft actually reflects the found content, not just believable-sounding text — check this by hand against a couple of test cases, not just "it ran without error."

**Your files after Step 4:**

```
project_3_documind_rag_agent/
├─ state.py
├─ graph.py                → search → answer sequence wired up
├─ retriever.py
├─ agents/
│   └─ retriever_agent.py
│   └─ reasoner_agent.py   → grounded-answer node
└─ nodes.py
```

## Step 5 — An Approval Agent That Only Asks a Human When It Really Needs To (final: 3 agents)

*Project: DocuMind-Smart-Documents-Agent-Ask-Your-Documents-Anything-Safely — Step 5 of 5: An Approval Agent That Only Asks a Human When It Really Needs To*

**What this step does:** adds the last specialist — a gate that automatically clears low-risk drafts, and only pauses for a real human when its own checks aren't enough — completing the full 3-agent team. **Why this step matters:** full automation is too risky for consequential actions, but a human check on every single run is too slow to actually be used — a gate that only escalates when it's genuinely unsure is the difference between a safety feature people keep and one they route around. **What's new vs. Step 4:** a new Approval node appears with an `interrupt()` gate; routing now branches based on the approval outcome. **What stays the same:** the Retriever and Reasoner from Steps 3-4 don't change — Approval only watches the Reasoner's draft, it doesn't rewrite it. **When you'll hit this for real:** any agent about to take a consequential, hard-to-undo action — sending something, spending money, deleting data — where full automation is too risky but a human-in-the-loop for every run is too slow. **Read first:** [09\\_langgraph Core Concepts](#) — "`interrupt()` : pausing for a human", [11\\_multi\\_agent\\_systems Core Concepts](#) — "Sequential" and "Supervisor" patterns.

**Stuck on this step?** [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Build the **Approval agent**: runs automatic checks on the Reasoner's draft (like — does it actually cite found content, does it avoid a "risky action" without support) — only calls `interrupt()` to ask a real human when its own checks aren't clear, or the action is genuinely important; otherwise it approves on its own and the graph continues. Write the auto-approval checks as an explicit checklist the code evaluates one item at a time, not one vague "is this okay?" model call — a checklist is what lets you point to exactly which check failed when a draft gets wrongly escalated, or wrongly waved through. 2. Wire the final routing: task → Retriever (if needed) → Reasoner → Approval → (approved on its own: done) or (interrupt: pause for a human) → resume → final answer. 3. Test the full cycle, plus the "search found nothing" path (Step 3), plus the "Approval agent approves on its own" path (not every run should need a human).

### Your files after Step 5 (final):

```
project_3_documind_rag_agent/
├─ state.py                → includes found chunks, not just the question
├─ graph.py                → full routing: search? → answer → approval → int
├─ retriever.py            → reused from 08_rag
├─ agents/
│   ├─ retriever_agent.py  → search + relevance-judging node
│   ├─ reasoner_agent.py   → grounded-answer node
│   └─ approval_agent.py   → automatic checks + interrupt() gate
└─ test_project3.py
```

### Final Deliverable: DocuMind-Smart-Documents-Agent-Ask-Your-Documents-Anything-Safely

— a 3-agent LangGraph system. A Retriever judges its own search quality. A Reasoner answers only from grounded material. An Approval agent auto-clears safe drafts and pauses for a human on the rest.

**Why this really is multi-agent (said plainly):** three separate roles, each a real node (or sub-graph) with its own job and its own pass/fail judgment — not one big node doing everything. There's still no supervisor here (routing is a fixed conditional path, not a dynamic `Command`-based router) — that dynamic routing is Project 4's job.

## Checklist Before You Call This Done

- [ ] `StateGraph` with typed state, at least one real conditional edge
- [ ] Checkpointer connected — state survives a pause/resume cycle
- [ ] Retriever agent only runs when needed, not always, and judges its own search results
- [ ] Reasoner agent produces answers grounded in what the Retriever actually found
- [ ] Approval agent approves low-risk drafts on its own, and only asks a human via `interrupt()` when needed — both paths tested

- [ ] Full search → answer → approve/pause → resume → grounded answer cycle works start to finish
- [ ] You can draw the graph from the code (or the other way around), without running it

Full requirements, test cases, and hints: [10\\_agent\\_workflows/README.md](#).

## Status

---

Not started. Track your progress in [../PROGRESS.md](#).

---

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**